Can a Multi-Agent Pipeline Effectively Simulate Human Reasoning in RAG Evaluation?

Aram Aytekin
uk097201@student.uni-kassel.de
University of Kassel
Kassel, Hessen, Germany

Abstract

The evaluation of Retrieval-Augmented Generation (RAG) systems is challenged by the divergence between automated metrics and human utility preferences. To address the high cost and scalability limits of human evaluation, this study investigates whether a multi-agent "Agent-as-a-Judge" pipeline can effectively simulate human reasoning. We propose a dimension-specific evaluation framework and compare two distinct architectures: a Pointwise Grading system that maps qualitative errors to numerical penalties, and a Pairwise Comparison system that relies on language-based, weight-of-evidence adjudication. Benchmarking these pipelines against a human gold standard using open-weight models, our empirical results demonstrate that standard Large Language Models are inherently poor at assigning absolute numerical grades, resulting in negative inter-rater reliability (Krippendorff's α). In contrast, leveraging the models' native linguistic capabilities through structured pairwise comparisons yields a 42% improvement over zero-shot baselines, peaking at $\alpha = 0.2449$ for Overall Quality. While our optimized multi-agent pipeline mechanically isolates flaws better than static judges, bridging the gap to the human consensus threshold ($\alpha = 0.80$) remains a formidable challenge.

CCS Concepts

• **Information systems** → **Evaluation of retrieval results**; • **Computing methodologies** → **Natural language generation**; *Multi-agent systems.*

Keywords

Retrieval-Augmented Generation, LLM-as-a-Judge, Multi-Agent Systems, System Evaluation, Automated Metrics

1 Introduction

The evaluation of Retrieval-Augmented Generation (RAG) systems represents a significant challenge in modern Information Retrieval. While RAG systems aim to provide direct, synthesized answers by grounding Large Language Model (LLM) outputs in retrieved documents, traditional evaluation metrics are increasingly reaching their limits. Standard n-gram overlap measures, such as BLEU [15] or ROUGE [12] often fail to capture the semantic nuances and factual accuracy required for reliable RAG output [8, 21]. Consequently, there is a growing discrepancy between automated scores and actual human utility preferences [6].

The core problem lies in the complexity of identifying and weighting specific errors. A minor stylistic inconsistency may be negligible, whereas a factual hallucination or a logical contradiction can render the entire response useless [9]. Currently, the evaluation of RAG systems is split between judgment-based paradigms, which rely on explicit utility scores, and reference-based paradigms, which compare outputs to ground-truth texts [6, 7].

In contrast, while human evaluation remains the "gold standard" for utility evidence, it is fundamentally hampered by high costs, low scalability, and the specialized expertise required for accurate assessment [6]. This creates a critical need for an automated "Agent-as-a-Judge" framework that can mimic human reasoning while maintaining the throughput of a machine-driven pipeline.

The methodology of this project focuses on a multi-agent evaluation pipeline designed to simulate human reasoning. The selection of these specific architectures is driven by both the ideal goals of evaluation and the inherent capabilities of current foundation models. A pointwise grading system intuitively comes to mind first, as it aligns with the aspiration of creating a "universal" evaluation framework—one where simply inputting a query and references yields a definitive, absolute grade across specific utility dimensions. On the other hand, directly comparing two responses might be the more natural and "easier" approach for LLMs. Because they are Large *Language* Models, they are fundamentally better equipped to handle linguistic nuances and relative qualitative comparisons than they are at assigning and adhering to pure numerical scales. To explore this dichotomy, we implement and compare two distinct judge architectures that utilize structured "agent judgments" as their primary evidence against human-validated data:

Approach 1: Pointwise Grading via Severity Assessment (Grade Agent). This method employs a two-stage workflow in which an initial agent identifies specific flaws within a response. A secondary agent then assesses these findings to assign a severity level to each identified issue. These severity-weighted labels are finally processed through mathematical models — specifically linear decay and exponential decay — to derive a final numerical score for the response.

Approach 2: Pairwise Comparison (Comp Agent) of Agent Judgments. This approach utilizes the same initial agent judgments as the previous method but replaces the scoring logic with a comparative framework. Rather than calculating an absolute grade, a comparison agent examines the sets of judgments for two competing responses and determines a winner based on a qualitative weight-of-evidence analysis.

To assess the reliability and validity of these automated approaches, we benchmark their outputs against the Crowd RAG Corpus 2025 (CrowdRAG-25) [6]. This dataset provides a robust human gold standard consisting of reliable utility evidence collected through extensive crowdsourcing.

The experimental framework utilizes a structured data format containing three core components for each instance: the query, the retrieved reference passages, and the generated or human written

responses. The performance of our automated judges is evaluated by comparing their results to thousands of pairwise human judgments. In these gold-standard evaluations, human raters performed direct comparisons between two responses across multiple utility dimensions to establish a definitive preference.

2 Related Work

The evaluation of Retrieval-Augmented Generation (RAG) has evolved from static n-gram metrics to dynamic, model-based assessment frameworks. This section outlines the shift from standard automated metrics to the "LLM-as-a-Judge" paradigm and the subsequent emergence of "Agent-as-a-Judge" architectures, which form the theoretical basis of our approach.

2.1 Automated RAG Evaluation

Traditional reference-based metrics such as BLEU or ROUGE have largely been superseded in the RAG domain due to their inability to capture semantic faithfulness and answer relevance [5]. Consequently, the field has adopted "Reference-Free" evaluation frameworks that utilize Large Language Models (LLMs) to approximate human judgment.

Prominent among these is RAGAS (Retrieval Augmented Generation Assessment) [5], which decomposes evaluation into component-wise metrics: *faithfulness*, *answer relevance*, and *context precision*. By prompting an LLM to verify if a generated claim can be inferred from the retrieved context, RAGAS provides a granular view of system performance without requiring ground-truth reference answers. Similarly, ARES (Automated RAG Evaluation System) [16] employs prediction-powered inference (PPI) and synthetic data generation to fine-tune lightweight judges, aiming to provide statistical confidence intervals for system ranking while reducing the computational cost of using large proprietary models for every evaluation. However, these frameworks typically treat the judge as a passive scorer processing a static input-output pair. They often lack the agency to investigate the "reasoning trajectory" of the system under evaluation, a limitation that recent agentic approaches seek to address.

2.2 From LLM-as-a-Judge to Agent-as-a-Judge

The LLM-as-a-Judge paradigm, popularized by benchmarks such as MT-Bench [22], posits that strong LLMs (e.g., GPT-4) can serve as reliable surrogates for human evaluators. While effective for general conversation, single-model judges often struggle with complex tasks requiring multi-step verification or domain-specific logic. This has led to the development of the Agent-as-a-Judge framework, where the evaluator is not merely a model but an autonomous agent capable of planning, tool use, and environmental interaction [23]. **Agentic Evaluation Frameworks:** Zhuge et al. [23] formally introduce the "Agent-as-a-Judge" concept, demonstrating that an agentic evaluator can monitor the entire "thought and action trajectory" of a candidate system. Unlike a static judge that sees only the final answer, an agentic judge can verify intermediate steps—such as checking if a generated code snippet actually compiles or if a retrieved document supports a specific sub-claim. Their experiments on the *DevAI* benchmark show that agentic judges achieve a 90% alignment with human consensus, significantly outperforming standard LLM judges (70%) [23].

This paradigm is particularly powerful for long-context and logic-heavy tasks. For instance, NarrativeFactScore [11] utilizes an agent to construct a "Character Knowledge Graph" (CKG) from long narratives (>100k tokens). By evaluating summaries against this structured graph rather than the raw text, the agent achieves a three-fold improvement in correlation with human judgments compared to standard LLM-based factuality metrics.
Furthermore, multi-agent architectures allow for "Judicial" or "Society-of-Agents" approaches. Mushtaq et al. [13] demonstrate this in the domain of Systematic Literature Reviews (SLRs), where a society of specialized agents (aligned with PRISMA guidelines) evaluates research protocols. Their system achieved an 84% agreement rate with expert reviewers, suggesting that decomposing evaluation into specialized agent roles (e.g., Methodologist, Statistician) yields higher reliability than a monolithic judge.

However, the reliability of such multi-agent systems is not absolute. Cemri et al. [2] highlight that multi-agent systems are prone to specific failure modes, such as "Inter-Agent Misalignment" and "Task Derailment." Their study on the MAST dataset underscores the necessity of robust system design, noting that failures often stem from coordination breakdowns rather than model capabilities alone. Our work builds upon these findings by implementing a judge that explicitly models these failure modes through structured penalty mechanisms.

3 Methodology

This section outlines the architectural design of the multi-agent pipelines developed to simulate human reasoning in RAG evaluation. We first present the general framework, focusing on the isolation of utility dimensions to ensure robust agent performance. Subsequently, we detail the specific implementation of the two proposed methodologies: the pointwise grading system and the pairwise comparison system.

3.1 General Pipeline Architecture

To address the complexity of identifying specific errors in RAG responses, we propose a modular, dimension-specific evaluation pipeline. Rather than relying on a monolithic agent to evaluate all aspects of a response simultaneously—a method prone to "context window saturation" and increased hallucination rates due to excessive input noise—we decompose the evaluation task into discrete, isolated processes. Our pipeline utilizes a "Divide and Conquer" strategy. For every instance (consisting of a query, retrieved context, and response), the system initiates independent evaluation cycles for each utility dimension defined in the CrowdRAG-25 corpus [6]. By isolating these dimensions, we ensure that the agents maintain a focused reasoning trajectory without the interference of conflicting evaluation criteria. The pipeline iterates through the following six dimensions, as seen in Table 1, processing them individually in both the Pointwise and Pairwise approaches.

This architectural decision serves two primary functions. First, it standardizes the input for our agents, ensuring that the *Grade Agent* and *Comp Agent* operate on identical dimensional definitions. Second, it minimizes the cognitive load on the LLM, as the agent

Table 1: Pairwise operationalization of utility dimensions proposed by Gienapp et al. [6], as well as overall quality.

Dimension	Question (Which response...)
Topical Correctness	... better answers the query?
Logical Coherence	... is easier to follow?
Stylistic Coherence	... has a more consistent writing style?
Broad Coverage	... looks at more aspects of the topic?
Deep Coverage	... explains things more in detail?
Internal Consistency	... is clearer about how different views fit together?
Overall Quality	... is better overall?

Note: This table is adapted from [6].

is prompted to reason about only one specific criterion at a time, thereby enhancing the reliability and explainability of the generated judgments.

To orchestrate the complex interactions between these independent evaluation cycles, we utilized n8n [18], a source-available workflow automation tool. This platform facilitated the visual design and execution of the agentic nodes, allowing for the modular integration of LLM inference with the structured data processing required for both severity assessments and pairwise comparisons.

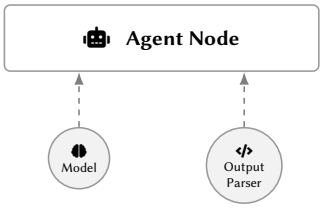

Figure 1: Structure of a single Agent Node in n8n

Figure 1 illustrates the internal composition of a standard agentic node within our workflow. Each node functions as a modular unit encapsulating a LangChain-based architecture. It connects the central agent logic (indicated by the robot icon) with a specific LLM configuration and a Structured Output Parser. Additionally, every agent is governed by distinct *System* and *User Prompts* that define its specific role and execution context. This setup ensures that the agent's reasoning is both powered by a capable model and constrained to a machine-readable format. The specific model architectures and parameters utilized for these nodes are detailed in Section Results.

3.2 Pointwise Grading System

In this approach, we implement a granular "Pointwise Grading" system designed to move beyond binary assessments of correctness. The architecture follows a sequential multi-agent workflow for each utility dimension, as illustrated in Figure 2.

The process begins with inserting the query, references and response from the CrowdRAG-25 [6] into an **Analyst Agent**, which scrutinizes the response to identify specific flaws or merits relative to the query and context. The output of this analysis is passed to a **Severity Classifier**, which assigns a categorical severity label to the identified issues. These qualitative labels are then mapped to numerical penalties via a **Deduction Map** module.

This workflow is executed in parallel for all six utility dimensions to ensure isolation. Once all dimension-specific scores are computed, they are merged into a single object. Finally, the **Overall Quality** score is calculated separately based on the aggregated weighted evidence of the individual dimensions.

The complete source code for this pipeline is available in our main repository [1], where the `agent-as-a-judge-grade.json` file in the backend directory encapsulates the entire evaluation workflow, including the system prompts and reasoning logic for all dimensions.

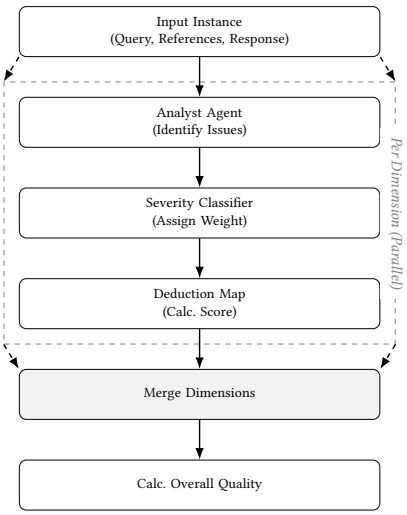

Figure 2: Workflow of the Pointwise Grading Pipeline.

3.2.1 The Analyst Agent. serves as the "detective" within our pipeline. Its primary function is not to assign a numerical score, but to identify, categorize, and substantiate specific flaws within the generated response.

Input Configuration. The agent operates on a strictly defined input set injected dynamically into the *User Prompt*. For every evaluation cycle, the agent receives:

(1) The original **Query**.
(2) The generated **Response** (raw text).
(3) The retrieved **References** (parsed and indexed).

The agent is explicitly instructed to ground its analysis solely on these provided references, strictly prohibiting the use of internal parametric knowledge to verify facts.

System Prompt Structure. To simulate a rigorous human evaluator, the *System Prompt* enforces a four-step reasoning process:

(1) **Holistic Review:** Assess the core informational needs of the query.
(2) **Factual Verification:** Cross-reference claims against the provided sources to detect hallucinations or contradictions.

(3) **Focus Analysis:** Identify irrelevant or misleading sections that deviate from the user's intent.
(4) **Omission Detection:** Scan references for critical information that was missing from the response.

The prompt constrains the agent to report findings using only four immutable categories: *Factual Error*, *Missing Information*, *Irrelevance*, and *Misleading/Focus Issues*.

Structured Output. To ensure the identified issues can be mathematically processed by the subsequent *Severity Classifier*, we utilize a Structured Output Parser. This forces the LLM to return a valid JSON object rather than unstructured text. The schema requires a summary and a list of specific error objects, as shown in Listing 1.

Listing 1: Analyst Agent Output Parser

```
{
  "analysis_summary": "A brief neutral summary of what the response did right and wrong.",
  "judgement_details": [
    {
      "type": "Factual Error",
      "statement": "The drug is safe for all ages.",
      "explanation": "Source [2] explicitly states it is fatal for children under 5.",
      "reference_index": [2]
    },
    {
      "type": "Missing",
      "explanation": "The response failed to mention the specific dosage of 50mg found in Source [1].",
      "reference_index": [1]
    },
    ...
  ]
}
```

While the technical scaffolding—dynamic input injection and structured output enforcement—remains constant across all six utility dimensions, the specific content within the *System Prompt Structure*, *Agent Output Parser* and the defined *Error Categories* are tailored to each specific criterion. Accordingly, the "Analysis Process" (e.g., the specific detection steps) is re-engineered to align with the unique requirements of the dimension. For instance, while *Topical Correctness* focuses on factual verification against references, dimensions like *Logical Coherence* necessitate a different analytical strategy, instructing the agent to scan for internal contradictions or fallacies within the argument structure independent of external sources.

3.2.2 The Severity Classifier. serves as the adjudication layer, translating the qualitative findings of the Analyst into quantitative metrics.

User Prompt Structure. The *User Prompt* for the Severity Classifier is standardized across all dimensions to ensure consistent context injection. It aggregates four key inputs:

(1) **Original Query:** The initial user intent.
(2) **Original Response:** The LLM's generated output.
(3) **Analyst Report:** The JSON list of identified issues (mapped to the `judgment_details` output from the previous step).
(4) **References:** (Conditional) Context retrieved from the knowledge base.

While the structure remains identical, the content of the *References* field is dynamic; it is populated for fact-based dimensions (e.g., *Topical Correctness*) but omitted for purely linguistic dimensions (e.g., *Stylistic Compliance*), where external grounding is unnecessary.

System Prompt: Classification Logic. The *System Prompt* employs a two-layer reasoning framework. The first layer, the **Classification Logic**, is universal across all six dimensions. It enforces a strict mapping of error intensity to a numerical score, ensuring that a "Level 2 Minor Error" carries the same mathematical weight in *Topical Correctness* as it does in *Logical Coherence*.

Table 2: Universal Classification Logic

Decision Step	Option	Definition
1. Category	Minor	Cosmetic or non-impeding issues.
	Moderate	Issues that degrade utility but preserve some value.
	Major	Critical failures making the response unusable.
2. Level	Level 1	*Low Intensity*: Bordering on the category below.
	Level 2	*Standard Intensity*: Classic example of this category.
	Level 3	*High Intensity*: Bordering on the category above.

System Prompt: Granular Severity Rubric. The second layer of the System Prompt is the **Granular Severity Rubric**. Unlike the logic, this section is highly tailored to the specific utility dimension. It maps the abstract Classification Logic to concrete error definitions relevant to that dimension.

For example, Table 3 illustrates a segment of the rubric used for *Topical Correctness*. The agent is provided with specific descriptions and examples for every possible error configuration, reducing ambiguity in the grading process.

Table 3: Excerpt from the Granular Severity Rubric for *Topical Correctness* for lowest vs. highest Category.

Category	Lvl	Description	Example
Minor	1	**Imprecise Paraphrasing.** Meaning shifted slightly but largely accurate.	*Source: "Rarely fatal"; Response: "Not often fatal."*
Major	3	**Dangerous Hallucination.** Inventing critical data (dosages, laws) not in sources.	*Response: "Take 500mg" (Source says 50mg).*

Structured Output Integration. To ensure seamless processing by the subsequent scoring logic, the Severity Classifier's qualitative assessment is rigidly formatted by a *Structured Output Parser*. Instead of generating free text, the parser enforces a schema that appends the determined `severity` category and numerical `severity_level` directly into the existing `judgement_details` array. This transforms the descriptive error list into a computable dataset, as shown in Listing 2.

Listing 2: Severity Classifier Output Parser

```
{
  "judgement_details": [
    {
      "severity": "Major",
      "severity_level": 2,
      "type": "Factual Error",
      "statement": "The drug is safe for all ages.",
      "explanation": "Source [2] explicitly states..."
      "reference_index": [2]
    },
    ...
  ]
}
```

3.2.3 Deduction Map and Scoring Logic. The final stage of the evaluation pipeline transforms the qualitative classifications from the Severity Classifier into a quantitative metric. This is achieved via a deterministic *Deduction Map*, which assigns a specific numerical penalty value to every unique combination of Severity Category and Severity Level. Crucially, this mapping is applied uniformly across all six utility dimensions, ensuring that errors of identical severity incur the same mathematical penalty regardless of the specific evaluation criterion.

The system calculates a cumulative "negative grade" by summing the penalty values of all identified issues. A higher total score indicates a higher density of defects and lower utility. The scoring progression follows a non-linear scaling philosophy designed to distinguish between nuisances and failures:

- **Linear Progression (Minor to Moderate):** For *Minor* and *Moderate* errors, the penalty values increase in a roughly linear fashion (see Table 4). This reflects that while these errors degrade the user experience (e.g., a missing nuance or a clunky sentence), they do not render the response fundamentally useless.
- **Exponential Escalation (Major):** The transition to the *Major* category introduces a steep, exponential increase in penalty values. This effectively acts as a "circuit breaker" mechanism. Specifically, a *Major Level 3* error is assigned a penalty of 100.0, representing a fatal flaw (e.g., a dangerous hallucination or total incoherence) that immediately invalidates the entire response regardless of other positive traits.

Table 4: The Deduction Map matrix

	Severity Level		
Category	**Level 1**	**Level 2**	**Level 3**
Minor	0.1	0.5	1.0
Moderate	2.5	4.0	6.0
Major	15.0	25.0	**100.0**

3.2.4 Overall Quality Aggregation. The final phase of the pipeline consolidates the isolated outputs from the six utility dimensions into a unified metric and a comprehensive error report.

Data Merging. First, the system aggregates the granular `judgement_details` arrays from all six evaluation agents. This results in a master list of all identified defects—ranging from factual errors to stylistic inconsistencies—providing the user with a holistic view of the response's flaws without losing the specific context of where those flaws originated.

Weighted Score Calculation. To compute the final `quality_overall` grade, the system employs a weighted aggregation strategy rather than a simple arithmetic mean. This approach acknowledges that not all dimensions contribute equally to the human perception of text utility.

We utilize the empirical weights derived from Gienapp et al. [6]. Specifically, we adopt the mean Kendall's τ correlation coefficients (as observed across all dimensions in their study) as the weighting factors. As detailed in Table 5, dimensions with a stronger correlation to human judgment of quality, such as *Topical Correctness* ($w = 0.42$), exert a proportionally greater influence on the final score than dimensions like *Logical Coherence* ($w = 0.28$).

The final score $Q_{overall}$ is computed as the weighted average of the individual dimension grades:

$$Q_{overall} = \frac{\sum_{d \in D} (G_d \cdot w_d)}{\sum_{d \in D} w_d} \tag{1}$$

Where D represents the set of six utility dimensions, G_d is the calculated grade for dimension d, and w_d is the corresponding empirical weight defined below.

Table 5: Weighting factors derived from mean Kendall's τ correlations in [6]. Higher weights indicate a stronger impact on the overall quality score.

Utility Dimension	Weight (w)
Topical Correctness	0.42
Deep Coverage	0.39
Internal Consistency	0.39
Broad Coverage	0.37
Stylistic Coherence	0.32
Logical Coherence	0.28

3.3 Pairwise Comparison System

While the Pointwise Grading pipeline establishes an absolute metric for individual responses, the *Pairwise Comparison System* is designed to determine relative superiority between two competing models. Rather than evaluating raw text from scratch, this system uniquely leverages the qualitative analytical data generated by the Pointwise pipeline, acting as an appellate judge reviewing prior evidence.

Figure 3 illustrates the workflow. While the final steps of merging the dimension-specific results and calculating the overall quality remain identical to the Pointwise Grading System, the evaluation phase differs significantly. The system ingests the outputs of two

distinct responses (Response a and Response b). Crucially, a preprocessing step filters out the numerical `grade` values, isolating only the textual `judgement_details` (the "Critique Reports"). This ensures the Adjudicator agent evaluates the nature and semantic impact of the errors natively, rather than simply comparing two pre-calculated numbers. The complete source code for this pipeline same as in the Grade System is available in our main repository [1], where the `agent-as-a-judge-comp.json` file in the backend directory encapsulates the entire evaluation workflow, including the system prompts and reasoning logic for all six dimensions.

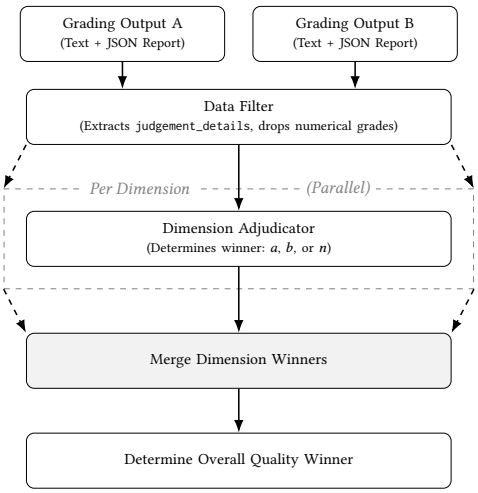

Figure 3: Workflow of the Pairwise Comparison Pipeline. The system ingests graded outputs, filters for textual critique, and dynamically adjudicates a winner across all utility dimensions.

3.3.1 Adjudicator Agent Prompt Architecture. The core of the pairwise system is the *Dimension Adjudicator* agent, which operates independently across all six utility dimensions.

User Prompt Structure. The *User Prompt* acts as the case file, presenting the raw data and the localized evidence. It injects the following components dynamically:

(1) **Ground Truth:** The original User Query and the relevant Reference Texts.
(2) **Evaluation of Response A:** The raw text of Response a alongside its specific Critique Report (the JSON array containing the type, statement, and explanation of identified errors).
(3) **Evaluation of Response B:** The raw text of Response b alongside its specific Critique Report.

System Prompt Structure. To prevent arbitrary decision-making, the *System Prompt* constrains the agent with a strict, rule-based decision logic termed the "Hierarchy of Failure." Similar to the Pointwise Grading pipeline, this logic is not static; the specific definitions and priorities within the hierarchy are heavily tailored to the unique requirements of each utility dimension. For instance, while a "Factual Error" represents the ultimate failure for *Topical Correctness*, a "Complete Breakdown of Syntax" serves as the equivalent fatal flaw for *Logical Coherence*.

Using *Topical Correctness* as an example, the agent is explicitly instructed not to merely count the number of errors, but to weigh their nature and impact according to the following precedence:

1. **The "Death Sentence" (Factual Errors):** Incorrect information is treated as the highest severity failure. If Response a contains a factual contradiction or hallucination, while Response b only contains omissions or irrelevant noise, Response b automatically wins.
2. **The "Substance" Test (Missing Information):** If factual correctness is tied, the system evaluates completeness. Missing a core sub-query is penalized heavier than missing a non-critical context detail.
3. **The "Noise" Test (Irrelevance / Focus):** Structural or focus issues (e.g., tangential sentences) act as tie-breakers. They determine the winner only if both responses are otherwise factually equivalent and equally comprehensive.
4. **The "Tie" Condition (n):** The agent is authorized to declare a tie (n) if both responses are flawless, or if both exhibit mathematically and semantically equivalent fatal flaws.

Structured Output Parser. Following the adjudication logic, the agent is forced to output its verdict via a strict JSON schema (Listing 3). It must explicitly declare the winning agent and provide a concise, objective explanation grounded in the Hierarchy of Failure.

Listing 3: JSON Schema for the Pairwise Adjudicator Output. The system mandates a definitive winner (a, b, or n) and a logical justification.

```
{
  "agent_winner": "a",
  "explanation": "Response a wins. Although a had a minor
       omission regarding secondary context, Response b
      contained a direct Factual Error regarding the
      specific date, which is a critical failure under
      the highest severity tier."
}
```

3.4 Frontend Application

To facilitate practical testing and provide an intuitive demonstration of the Pairwise Comparison System, we developed a dedicated web-based frontend. This interface seamlessly connects to the n8n backend, allowing users to interact directly with the multi-agent pipeline without requiring programming expertise. The complete frontend can be found in our repository [1].

Data Input and Formatting. Users can initiate a comparison through two distinct input methods. For quick, custom tests, users can manually populate the query, reference texts, and both competing responses into the designated text fields (Figure 4). Alternatively, for batch testing or reproducing specific dataset entries, the interface supports uploading JSON files (Figure 5). These files must be formatted according to the CrowdRAG corpus schema [6] and both uploaded responses must share the exact same underlying reference texts.

The reference texts can be toggled between a structured "List View" and a "Raw Text" schema (Figure 6), giving the user flexibility in how context is inserted.

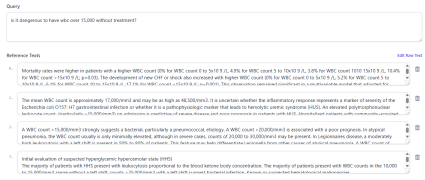

Figure 4: Manual input interface for queries and references.

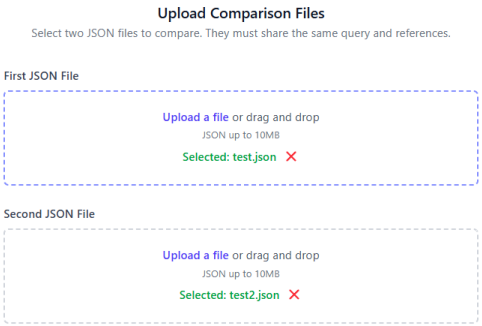

Figure 5: JSON upload module for CrowdRAG-formatted data.

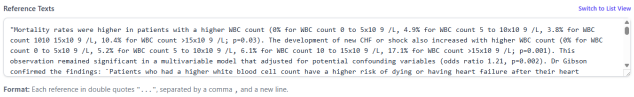

Figure 6: The Reference Texts module, allowing users to switch between Raw Text and List View formats.

Interactive Citation Tracking. A core feature of the response input fields is the automated citation tracking. The frontend scans the raw text of both responses for bracketed citation markers (e.g., `[1, 2]`). Valid citations that correspond to an existing reference index are highlighted in blue and function as clickable hyperlinks, allowing the user to instantly scroll up to the respective source text (Figure 7). Conversely, if an LLM hallucinates a citation that does not exist in the provided context (e.g., `[308]`), the frontend immediately flags it in red (Figure 7), providing an instant visual indicator of a factual or formatting failure.

Execution and Analytical Output. Upon clicking "Submit Comparison," the frontend triggers the backend Pairwise pipeline. During the inference phase, a loading animation indicates that the Analyst Agents are actively processing the responses across all utility dimensions (Figure 8).

Once the initial analysis is complete, the UI populates the "Analysis Results" section. This area provides an expandable dropdown menu for every utility dimension for both Response A and Response B (Figure 9). Expanding a dimension reveals the localized Critique Report, detailing the Overall Judgement and highlighting the specific "Key Issues" (e.g., Irrelevance, Missing Information) identified by the Analyst Agent (Figure 10).

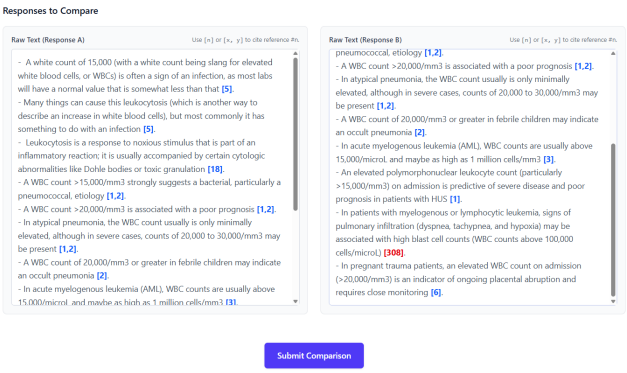

Figure 7: Valid citations are highlighted in blue and act as clickable links to the source. Hallucinated or invalid citations are immediately flagged in red.

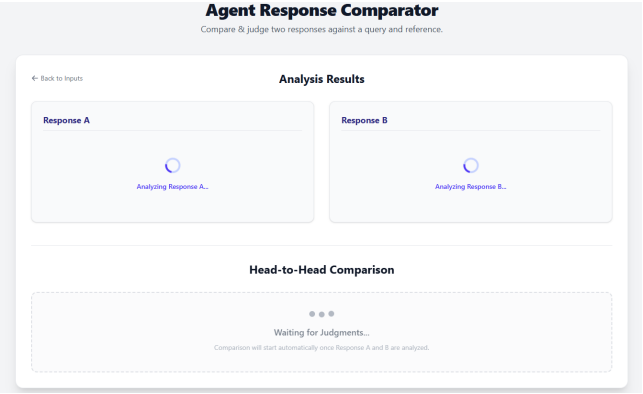

Figure 8: Loading interface during the backend agent processing phase.

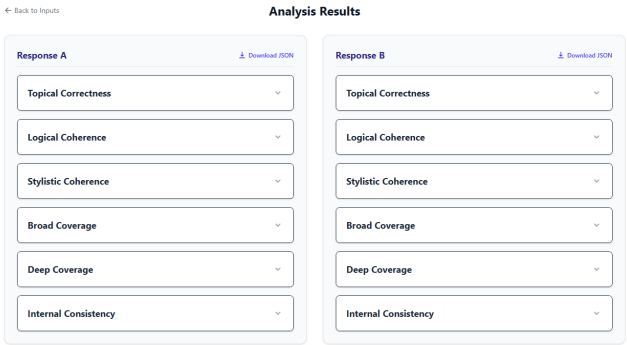

Figure 9: Overview of the completed Analysis Results for both responses.

The Final Verdict. In the final step, the textual evidence displayed in the Analysis Results is automatically forwarded to the Comparator Agent. The frontend updates to present the "Head-to-Head Comparison" (Figure 11). This conclusive dashboard prominently

displays the Overall Quality winner alongside the numerical score approximations. Below the final verdict, a detailed breakdown reveals which response won each specific utility dimension and provides the Adjudicator's logical explanation for the decision, ensuring complete transparency of the pipeline's reasoning.

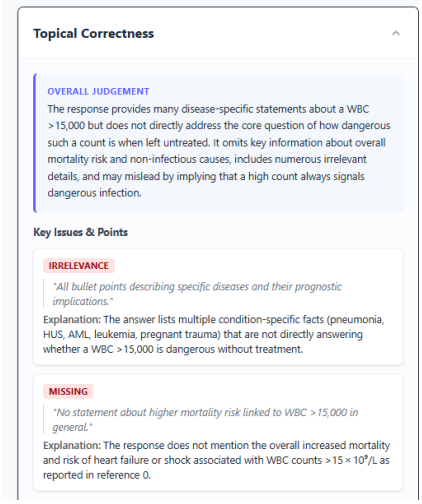

Figure 10: Expanded view of the Analysis Results, showing the extracted Key Issues and specific flaw categorizations.

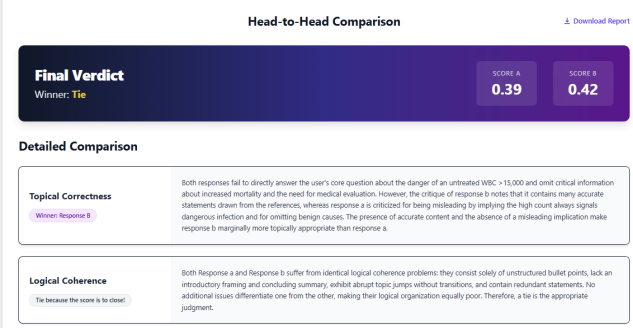

Figure 11: The Final Verdict dashboard, displaying the overall winner, score approximation, and detailed adjudicator reasoning per dimension.

4 Experimental Setup

To rigorously evaluate the efficacy of our Agent-as-a-Judge pipelines (Pointwise Grading and Pairwise Comparison), we conduct an empirical evaluation against a human-annotated gold standard.

4.1 Dataset and Ground Truth

Our evaluation is grounded in crowdsourced human judgment data derived from prior efforts by Gienapp et al. [6]. The dataset consists of user queries, reference documents, multiple LLM-generated responses, and corresponding human ratings. While the dataset is inherently scoped to 65 distinct topics, it contains a larger volume of individual response pairs per topic.

To comprehensively assess our Agent-as-a-Judge pipelines, we evaluate our system against the human gold standard under two distinct conditions:

Unfiltered (All Ratings): The complete set of response pairs and their aggregated human ratings across all 65 topics.

Majority Filter (Minimally Differentiable Items): A rigorously filtered subset designed to isolate pairs where a clear human consensus is absent. We analyze the annotator voting behavior to identify minimally differentiable item pairs. A "gold standard" tie (neutral) label is assigned only when a pair lacks a clear majority vote (e.g., at least 3 out of 5 annotators agreeing) across a majority of the utility dimensions (4 out of 7).

For instance, under the majority filter condition, an annotator distribution of `["A", "B", "N", "A", "N"]` for the *Topical Correctness* dimension lacks an absolute majority for either model. Consequently, the human gold label for this specific pair and dimension is established as `"n"` (tie).

4.2 LLM Selection

To assess whether the underlying foundation model significantly impacts the reasoning and grading capabilities of the Agent-as-a-Judge pipeline, we evaluate our system using two distinct, large-scale open-weight models:

(1) **120B_GPT_OSS**: A massive 120-billion parameter open-source model designed for complex reasoning tasks [14].
(2) **Qwen3-235B**: A state-of-the-art model from Alibaba with an extended context window, allowing for thorough processing of the dense user prompts and reference texts [19].

By running both the Grading and Comparison pipelines through these models, we can isolate the effectiveness of our prompt engineering and workflow architecture from the raw zero-shot capabilities of the LLMs themselves.

4.3 Evaluation Metrics

To measure the alignment between our AI adjudicators and the human gold standard, we employ Krippendorff's α [17], a robust statistical measure for inter-rater reliability that accounts for chance agreement. According to standard conventions, an α value of 0.80 or higher is generally considered to indicate strong and reliable agreement with the human judgments [17].

The objective of our experimental setup is to demonstrate that a structured, multi-agent pipeline outperforms standard zero-shot LLM evaluation. Therefore, our primary benchmark is the zero-shot LLM-to-Human agreement scores established by Gienapp et al. [6]. Table 7 details these baseline α scores across all utility dimensions. Our goal is to significantly exceed these thresholds using both our Pointwise and Pairwise approaches.

The complete Python source code for computing the Krippendorff's α metrics against the dataset, including handling for dynamic thresholds and API routing, is publicly available in our repository [1].

4.4 Execution Configurations and Prompt Variations

To thoroughly evaluate the robustness of our pipeline and the specific impact of our prompt engineering, we tested both the Pointwise Grading and Pairwise Comparison systems under multiple execution configurations.

Pointwise Grading Execution. In the Pointwise system, the pipeline independently calculates a numerical penalty `grade` for Response a and Response b. To compare these grades against the categorical human gold standard (`a`, `b`, or `n`), we establish a numerical threshold mechanism:

- **Thresholding:** If the absolute difference between the grades is less than or equal to a defined threshold ($|Grade_a - Grade_b| \leq t$), the system predicts a tie (`n`). Otherwise, the response with the lower penalty grade is declared the winner. We empirically tested multiple threshold values to identify the optimal sensitivity for distinguishing model outputs.
- **Prompt Ablation (Zero-Shot Analyst):** To also test the necessity of the structured reasoning steps defined in the system promt of the Analyst Agent, we ran a "zero-shot" ablation. In this configuration, the rigid four-step reasoning process (Holistic Review, Factual Verification, Focus Analysis, Omission Detection) is completely removed. The Analyst agent is instead given a basic, unguided instruction to act as a "detective" and extract flaws based solely on the query and references, without specific analytical guardrails.

Pairwise Comparison Execution. Unlike the Grading system, the Pairwise Comparison pipeline directly outputs a predicted winner (`a`, `b`, or `n`), which is immediately evaluated against the human gold standard. To isolate the variables driving the Adjudicator's performance, we tested four distinct configurations:

(1) **Standard Setup:** The full, comprehensive pipeline utilizing the detailed "Hierarchy of Failure" decision logic and all available contextual inputs.
(2) **Zero-Shot:** We removed the rigorous error-weighing logic (e.g., "The Death Sentence", "The Substance Test"). The agent receives only its role definition, the input data, and a basic instruction to declare a winner or a tie, forcing it to rely on its internal zero-shot reasoning to weigh the severity of the provided Critique Reports.
(3) **No References:** We preserved the full System Prompt logic but artificially blinded the Adjudicator by removing the original Reference Texts from the User Prompt. The agent must adjudicate solely based on the text of the responses and the Analyst's Critique Reports.
(4) **Zero-Shot + No References:** The most restrictive ablation, combining both the removal of the specific "Hierarchy of Failure" logic and the omission of the Reference Texts.

5 Results and Analysis

In this section, we present the empirical results of our Agent-as-a-Judge pipelines, comparing the Pointwise Grading and Pairwise Comparison systems against the human gold standard. We evaluate the performance using Krippendorff's α and analyze the underlying reasons for the observed misalignments between LLM and human judgments. All graphics and detailed data for all experimental configurations are available in our repository [1].

5.1 Quantitative Results: Pipeline Performance

To establish the efficacy of our approaches, we iteratively tested various configurations, comparing the resulting α_{total} — The arithmic mean of across all dimensions — scores against the zero-shot LLM baseline established in prior literature.

Pointwise Grading Performance. The Grading pipeline calculates the absolute penalty difference between two responses, classifying a tie if the difference falls within a specific threshold t. We empirically evaluated thresholds of $t \in \{4.0, 4.5, 5.0, 7.5\}$. The results were uniformly poor, yielding negative total α scores across all settings. The highest agreement was found at the strictest threshold ($t = 4.0$), resulting in $\alpha_{total} = -0.0646$. As the threshold increased, the agreement deteriorated significantly (e.g., $t = 7.5$ yielded $\alpha_{total} = -0.2677$). Due to this systemic underperformance, the Grading system was exclusively evaluated using the OSS model [14] without the majority filter. Consequently, we elected not to test the Zero-Shot Analyst ablation within the Pointwise framework. Instead, we applied this zero-shot configuration exclusively to the Pairwise Comparison pipeline to investigate whether unstructured analyst judgments might paradoxically lead to improved comparative results.

Pairwise Comparison Performance. The Pairwise Comparison pipeline demonstrated superior, albeit still limited, alignment with human judgments. We conducted an ablation study using the OSS model [14] to isolate the impact of different prompt structures, evaluating both the unfiltered dataset and the rigorously filtered majority-vote dataset. As expected, the majority filter consistently yielded higher agreement:

Table 6: Comparison of α_{total} across different setups.

Setup	$\alpha_{total_{Unfiltered}}$	$\alpha_{total_{Majority}}$
Full Setup	0.0760	0.0857
No References	0.0720	0.0818
Zero-Shot	0.0965	**0.1090**
Zero-Shot + No Ref	0.0984	0.1070

Since the restrictive "Zero-Shot" ablation surprisingly proved to be among the most effective (for majority), we applied this specific configuration to the more capable Qwen model [19]. By combining the Qwen model with a Zero-Shot Analyst (for generating the Critique Reports) and the Zero-Shot Adjudicator, we achieved our peak performance: α_{total} = **0.1354** on the majority-filtered data.

Comparative Overview. Table 7 details the α scores for all utility dimensions across our tested configurations. To highlight the impact of our best pipeline (Qwen Optimized), we provide the percentage variance compared to the established baseline. While our optimal setup achieves a **42%** overall improvement and massive gains in previously near-zero dimensions like *Deep Coverage* (+1728%) and *Overall Quality* (+308%), it still suffers losses in *Topical Correctness* and *Internal Consistency*.

Table 7: Krippendorff's α across utility dimensions for various pipeline configurations compared to the baseline. Abbreviations: TC (Topical Correctness), LC (Logical Coherence), SC (Stylistic Coherence), BC (Broad Coverage), DC (Deep Coverage), IC (Internal Consistency), OQ (Overall Quality), Total. *Note: All Comp/Qwen rows utilize the majority-filtered data.*

Setup	TC	LC	SC	BC	DC	IC	OQ	Total
Baseline	**0.1000**	**0.1600**	**0.1400**	**0.080**	**0.0100**	**0.1200**	**0.0600**	**0.0957**
Grade (T=4.0)	0.0303	−0.0042	−0.1568	−0.0213	−0.1121	−0.0109	−0.1771	**-0.0646**
Comp Full	0.0576	0.0302	0.0944	0.0624	0.1401	0.0314	0.1841	**0.0857**
Comp No Ref	0.0440	0.0111	0.0988	0.0883	0.1130	0.0175	0.2001	**0.0818**
Comp ZS	0.0978	−0.0082	0.0996	0.1406	0.1620	0.0424	0.2290	**0.1090**
Comp ZS + No Ref	0.0876	−0.0090	0.0950	0.1547	0.1548	0.0359	0.2299	**0.1070**
Qwen Optimized	**0.0698**	**0.0958**	**0.1411**	**0.1597**	**0.1828**	**0.0534**	**0.2449**	**0.1354**
% vs. Baseline	−30%	−40%	+1%	+100%	+1728%	−55%	+308%	***+42%***

5.2 Analysis

The empirical data indicates that while multi-agent architectures can surpass zero-shot baselines in specific utility dimensions, reliable, human-aligned RAG evaluation remains exceptionally difficult. An objective assessment of the absolute metrics necessitates a highly critical and defensive interpretation: while our optimized pipeline yields relative improvements, the maximum agreement achieved across any configuration is merely $\alpha = 0.2449$ for *Overall Quality* (Qwen Optimized). When compared to the established threshold of $\alpha = 0.80$ required for strong inter-rater reliability, it is evident that our automated adjudicators are still far from consistently replicating human consensus.

5.2.1 Topic Correlation and Variance. A deeper analysis of the results reveals a severe lack of cross-dimensional correlation within individual topics. Figure 12 visualizes the α scores for Topics 27 through 39 under the Zero-Shot configuration. The highly erratic, crisscrossing trajectories indicate that an LLM might perfectly align with human judgment regarding *Broad Coverage* for a specific topic, but completely fail to assess *Logical Coherence* accurately for the exact same text. Furthermore, the dataset contains an extremely low number of "Stable" topics (instances where human agreement was highly consistent across all dimensions), often resulting in zero or only one stable topic per test scenario.

We also observed extreme variance depending on the dimension being evaluated. As illustrated in the supplementary bar charts (see repository for full documentation), *Overall Quality* acts as a relatively robust metric across most tests, though as noted, it still peaks at a drastically insufficient 0.2449. Conversely, *Stylistic Coherence* proved to be a catastrophic failure point for the Pointwise Grading system, consistently generating deep negative α scores (e.g., −0.1568), indicating that the mathematical penalty mapping completely inverted actual human stylistic preferences.

The stark contrast in performance across dimensions highlights a clear dichotomy in LLM evaluation capabilities: the distinction between macro-level semantic processing and micro-level logical reasoning. The dimensions that saw the most significant improvements—*Broad Coverage*, *Deep Coverage*, and *Overall Quality*—rely primarily on holistic, macroscopic assessments. LLMs naturally excel at pattern recognition and semantic matching, making them highly capable of determining whether one response encompasses more sub-topics (Broad Coverage) or provides a higher density of detail (Deep Coverage) than another. Furthermore, because human raters often heavily weigh comprehensiveness when determining *Overall Quality*, the LLMs' inherent bias toward detailed, lengthy responses aligns closely with human preferences in this regard. Conversely, the dimensions where the pipeline struggled—*Topical Correctness*, *Logical Coherence*, and *Internal Consistency*—demand rigorous, microscopic logical tracking. Detecting subtle factual deviations from the reference texts (Topical Correctness) or identifying internal contradictions (Internal Consistency) requires a level of precise, multi-step deduction that zero-shot LLM evaluators still struggle to consistently maintain. While a human judge might immediately penalize a response for a single factual hallucination, the LLM adjudicator may mistakenly overlook the localized error if the surrounding text appears highly fluent and comprehensive [3].

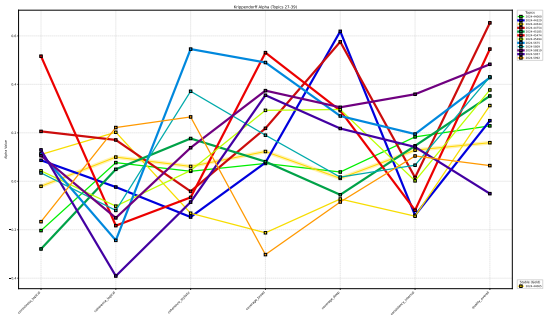

Figure 12: Krippendorff's α trajectories for Topics 27-39 [1] (Zero-Shot without References). The chaotic crossing of lines highlights the lack of intra-topic consistency across different utility dimensions.

5.2.2 The Misalignment of Human and LLM Reasoning. The persistent difficulty in achieving high agreement scores raises a fundamental question regarding the nature of human reasoning versus machine evaluation. Recent scientific consensus suggests that LLM-to-human agreement often shows only little overlap with gold labels, calling into question the absolute correctness of LLM judgments [6].

This misalignment stems from inherent differences in how humans and LLMs process and structure information. Human and LLM writing behaviors differ in several critical aspects [6]. For instance, humans tend to segment information into smaller units with precise document attribution, whereas LLMs favor broader statements that synthesize multiple documents abstractly [6]. Furthermore, humans maintain a consistent, accessible reading ease (typically high-school level), while LLMs systematically generate more complex, university-level text when synthesizing sources [6].

When humans evaluate a text, these subtle linguistic preferences—granularity, accessibility, and structural familiarity—unconsciously drive their utility judgments. In contrast, LLMs evaluating the same text often fail to differentiate these fine-grained dimensions, instead detecting only a generalized preference [6]. Consequently, while our Agent-as-a-Judge pipeline successfully isolates errors mechanically, bridging the gap to replicate the subjective, holistic nuances of human preference remains a formidable challenge.

6 Conclusion & Future Work

This study investigated whether a multi-agent pipeline could effectively simulate human reasoning in the evaluation of Retrieval-Augmented Generation (RAG) systems. By comparing a Pointwise Grading architecture against a Pairwise Comparison framework, we sought to determine the optimal method for leveraging LLMs as automated judges.

Our empirical results demonstrate a stark contrast between the two approaches. The Pointwise Grading system, which attempted to map qualitative errors to a strict numerical penalty matrix, failed significantly, yielding negative agreement scores across all tested thresholds. This systemic failure highlights a fundamental limitation in current foundation models: LLMs are inherently poor at assigning absolute numerical grades. Because their underlying architecture treats numerical outputs as discrete classification tokens rather than continuous regression variables, they struggle to apply mathematical penalty scales consistently without specialized regression-aware fine-tuning [4].

Conversely, the Pairwise Comparison system, which allowed the Adjudicator agent to evaluate based purely on textual Critique Reports, yielded substantially better alignment with the human gold standard. By replacing arbitrary numerical constraints with a language-based weight-of-evidence analysis, the LLM was better able to utilize its native linguistic reasoning capabilities. Furthermore, our experiments clearly indicate that the raw reasoning capacity of the underlying foundation model is a critical factor in this domain; utilizing a stronger, larger-scale model like Qwen tangibly improved the quality of the adjudications and resulted in our highest performance. However, even this optimized Qwen-based pairwise configuration peaked at an agreement of $\alpha_{QO} = 0.2449$, indicating that bridging the gap to the human consensus threshold ($\alpha = 0.80$) remains a formidable challenge.

Future Work. Given the catastrophic failure of the numerical grading pipeline, we strongly recommend that future RAG evaluation frameworks abandon absolute numerical scoring in favor of purely language-based comparative paradigms. Because LLMs are fundamentally language models, their evaluative capacity is maximized when framing judgments as semantic, text-to-text comparisons rather than scalar assignments [4].

However, relying on language-based pairwise comparisons introduces its own set of challenges, particularly regarding systemic biases such as position bias, verbosity bias, and fallacy oversight. To mitigate these issues, future research should explore advanced prompting and reasoning structures. Recent theoretical work suggests that integrating pointwise reasoning *within* a pairwise framework—where the LLM first generates independent textual critiques for each response before executing the final comparative judgment—can significantly reduce these adversarial biases [10]. Additionally, developing adaptive, debiased policies that explicitly train the judging behavior to maintain cross-mode consistency will be critical for scaling Agent-as-a-Judge pipelines to match human reliability [20].

References

[1] Aram Aytekin. 2025. *Agent-as-a-Judge Repository.* https://github.com/CuzImAram/agent-as-a-judge Source code for frontend and backend implementations. Accessed: 2026-02-01.

[2] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657 [cs.AI] https://arxiv.org/abs/2503.13657

[3] Guiming Hardy Chen, Shunian Chen, Ziche Liu, Feng Jiang, and Benyou Wang. 2024. Humans or LLMs as the Judge? A Study on Judgement Biases. arXiv:2402.10669 [cs.CL] https://arxiv.org/abs/2402.10669

[4] Cheng-Han Chiang, Hung yi Lee, and Michal Lukasik. 2025. TRACT: Regression-Aware Fine-tuning Meets Chain-of-Thought Reasoning for LLM-as-a-Judge. arXiv:2503.04381 [cs.CL] https://arxiv.org/abs/2503.04381

[5] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2025. Ragas: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217 [cs.CL] https://arxiv.org/abs/2309.15217

[6] Lukas Gienapp, Tim Hagen, Maik Fröbe, Matthias Hagen, Benno Stein, Martin Potthast, and Harrisen Scells. 2025. The Viability of Crowdsourcing for RAG Evaluation. In *48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2025).* ACM, 159 – 169. doi:10.1145/3726302.3730093

[7] Lukas Gienapp, Harrisen Scells, Niklas Deckers, Janek Bevendorff, Shuai Wang, Johannes Kiesel, Shahbaz Syed, Maik Fröbe, Guido Zuccon, Benno Stein, Matthias Hagen, and Martin Potthast. 2024. Evaluating Generative Ad Hoc Information Retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2024).* ACM, 1916–1929. doi:10.1145/3626772.3657849

[8] Tanya Goyal, Junyi Jessy Li, and Greg Durrett. 2023. News Summarization and Evaluation in the Era of GPT-3. arXiv:2209.12356 [cs.CL] https://arxiv.org/abs/2209.12356

[9] Tom Hosking, Phil Blunsom, and Max Bartolo. 2024. Human Feedback is not Gold Standard. arXiv:2309.16349 [cs.CL] https://arxiv.org/abs/2309.16349

[10] Hawon Jeong, ChaeHun Park, Jimin Hong, Hojoon Lee, and Jaegul Choo. 2025. The Comparative Trap: Pairwise Comparisons Amplifies Biased Preferences of LLM Evaluators. arXiv:2406.12319 [cs.CL] https://arxiv.org/abs/2406.12319

[11] Yeonseok Jeong, Minsoo Kim, Seung won Hwang, and Byung-Hak Kim. 2025. Agent-as-Judge for Factual Summarization of Long Narratives. arXiv:2501.09993 [cs.CL] https://arxiv.org/abs/2501.09993

[12] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out.* Association for Computational Linguistics, Barcelona, Spain, 74–81. https://aclanthology.org/W04-1013/

[13] Abdullah Mushtaq, Muhammad Rafay Naeem, Ibrahim Ghaznavi, Alaa Abdalrazaq, Aliya Tabassum, and Junaid Qadir. 2025. Can Agents Judge Systematic Reviews Like Humans? Evaluating SLRs with LLM-based Multi-Agent System. arXiv:2509.17240 [cs.AI] https://arxiv.org/abs/2509.17240

[14] OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrylov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado,

Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily, Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 [cs.CL] https://arxiv.org/abs/2508.10925

[15] Matt Post. 2018. A Call for Clarity in Reporting BLEU Scores. arXiv:1804.08771 [cs.CL] https://arxiv.org/abs/1804.08771

[16] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. arXiv:2311.09476 [cs.CL] https://arxiv.org/abs/2311.09476

[17] Wikipedia. 2026. Krippendorff's alpha — Wikipedia, The Free Encyclopedia. http://en.wikipedia.org/w/index.php?title=Krippendorff's%20alpha&oldid=1335776817. [Online; accessed 22-February-2026].

[18] Wikipedia contributors. 2024. n8n — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/N8n [Online; accessed 01-February-2026].

[19] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[20] Bo Yang, Lanfei Feng, Yunkui Chen, Yu Zhang, Xiao Xu, and Shijian Li. 2026. FairJudge: An Adaptive, Debiased, and Consistent LLM-as-a-Judge. arXiv:2602.06625 [cs.CL] https://arxiv.org/abs/2602.06625

[21] Tianyi Zhang, Faisal Ladhak, Esin Durmus, Percy Liang, Kathleen McKeown, and Tatsunori B. Hashimoto. 2023. Benchmarking Large Language Models for News Summarization. arXiv:2301.13848 [cs.CL] https://arxiv.org/abs/2301.13848

[22] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685 [cs.CL] https://arxiv.org/abs/2306.05685

[23] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. 2024. Agent-as-a-Judge: Evaluate Agents with Agents. arXiv:2410.10934 [cs.AI] https://arxiv.org/abs/2410.10934